pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

Slopes and Butterflies

WEEK 2 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: five shapes, each with its sign

The Desk Lead's Note

The morning note describes the curve in shorthand: 2s10s, 5s30s, 3m10y, and two flies. Give me all five for 2 October 2026 with the sign of each written down, so nobody reads one backwards, and put them in "marketdata", tied to Excel. Check our 2s10s against FRED's own. Say where 2s10s and the 2s5s10s fly sit in their history. Give me the face amounts at which a 2s5s10s fly's three DV01s net to zero, from your "bondmath". Last, keep 2026's slopes in a file under Git, so a rerun that moves a number shows up. Describe the shape; never say what it means for what comes next.

Every number today comes from the Treasury file; the one live cell only checks it.

```
Work folder pfi_course3_09_k4w7q2xr: 3 data files, Course 2's bondmath.py, marketdata.py as notebook 08 left it, 8 commits, tagged week1
.................................   [100%]
35 passed in 0.50s
```

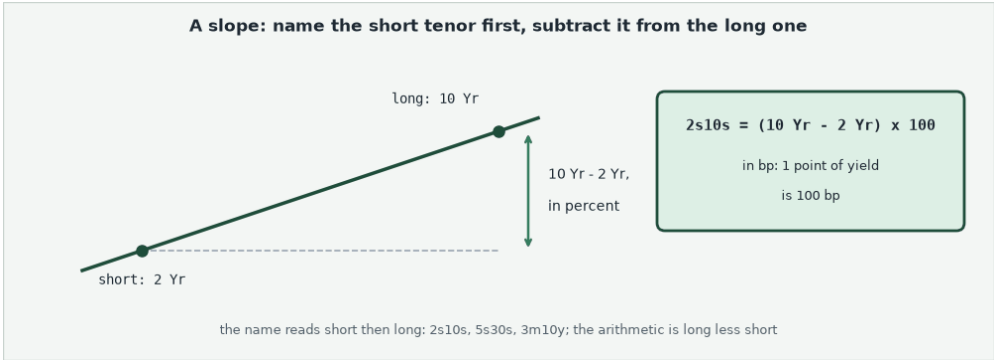
Imports and the Treasury File

```
treasury = pd.read_csv('treasury_par_yields.csv', parse_dates=['date'], index_col='date',
                       float_precision='round_trip')
tenors = ['3 Mo', '2 Yr', '5 Yr', '10 Yr', '30 Yr']
print(f'{len(treasury):,} market days; on 2026-10-02, in percent:')
print(treasury.loc['2026-10-02', tenors].to_dict())
2,940 market days; on 2026-10-02, in percent:
{'3 Mo': 4.19, '2 Yr': 4.83, '5 Yr': 5.06, '10 Yr': 5.28, '30 Yr': 5.63}
```

Notebook 08 said where a level sits; today the curve's shape gets its numbers.

9.1 Slopes: 2s10s, 5s30s, 3m10y, with signs

The note's shorthand names two tenors, the short one first.



A slope is the long tenor's yield less the short one's, times 100 for bp.

2s10s by Hand

```
two_tens_bp = (treasury.loc['2026-10-02', '10 Yr'] - treasury.loc['2026-10-02', '2 Yr']) * 100
print(f'2s10s on 2026-10-02: {two_tens_bp:.2f} bp')
2s10s on 2026-10-02: 45.00 bp
```

":.2f" hides notebook 06's binary tail on the stored number.

Excel side by side. The Treasury file opened in Excel: dates in A, the 3 Mo in E, the 2 Yr in I, the 5 Yr in K, the 10 Yr in M, the 30 Yr in O. J and L hold the 3 Yr and 7 Yr, so a letter off by one is a different tenor.

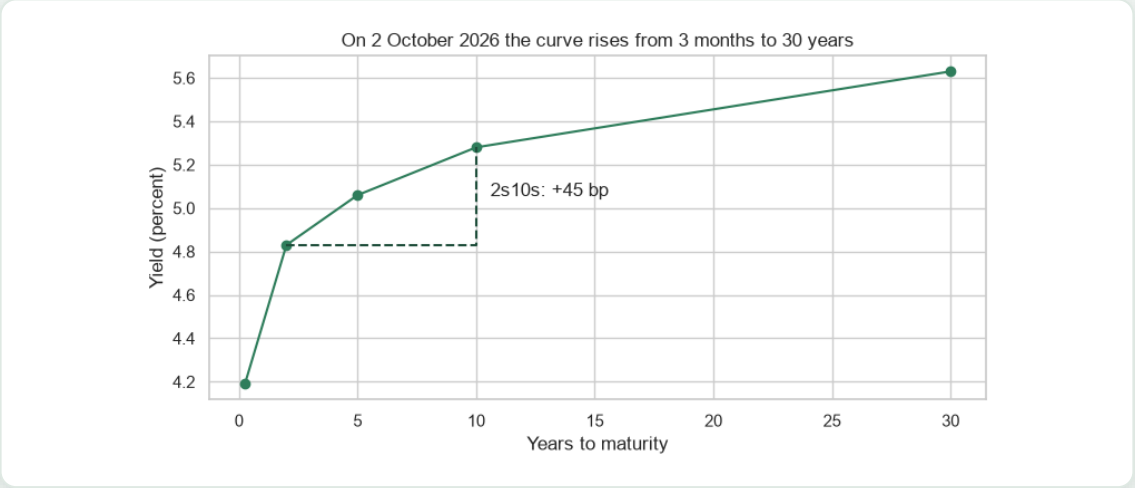
Job	Python	Excel	Result
2s10s on the last row	`two_tens_bp`	`=(M2941-I2941)*100`	45
2s10s on every row	`the next cell`	`=(M2-I2)*100 filled to row 2941`	equal on all 2,940 rows

Three slopes, every date

```
slopes_by_hand = pd.DataFrame({
    '2s10s': (treasury['10 Yr'] - treasury['2 Yr']) * 100,
    '5s30s': (treasury['30 Yr'] - treasury['5 Yr']) * 100,
    '3m10y': (treasury['10 Yr'] - treasury['3 Mo']) * 100,
})
slopes_by_hand.tail(3).round(2)
```

date	2s10s	5s30s	3m10y
2026-09-30	41.0	55.0	109.0
2026-10-01	46.0	60.0	107.0
2026-10-02	45.0	57.0	109.0

The Curve on 2 October 2026



Excel Side by Side

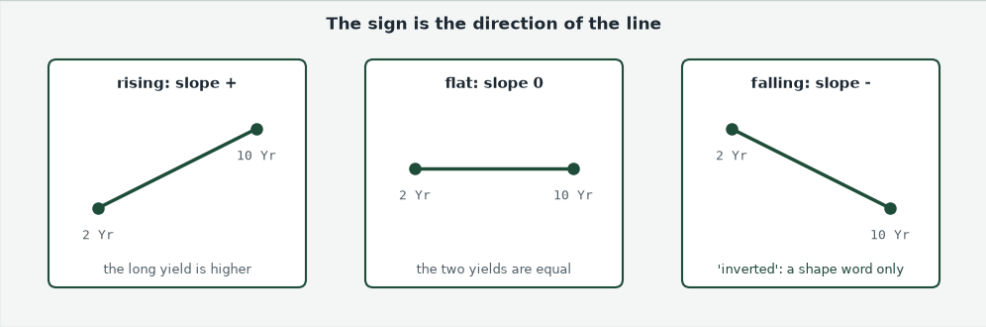
Job	Python	Excel	Result, last row
5s30s	`slopes_by_hand['5s30s']`	`=(O2-K2)*100 filled down`	57
3m10y	`slopes_by_hand['3m10y']`	`=(M2-E2)*100 filled down`	109

Excel's columns equal these on all 2,940 rows.

The lowest 2s10s, and what a sign means

Q. When was 2s10s lowest, and what were the two yields?

```
lowest_date = slopes_by_hand['2s10s'].idxmin()
print(f"lowest 2s10s: {slopes_by_hand['2s10s'].min():.2f} bp on {lowest_date:%Y-%m-%d}")
print(f"2 Yr {treasury.loc[lowest_date, '2 Yr']:.2f}, "
      f"10 Yr {treasury.loc[lowest_date, '10 Yr']:.2f}")
lowest 2s10s: -108.00 bp on 2023-07-03
2 Yr 4.94, 10 Yr 3.86
```

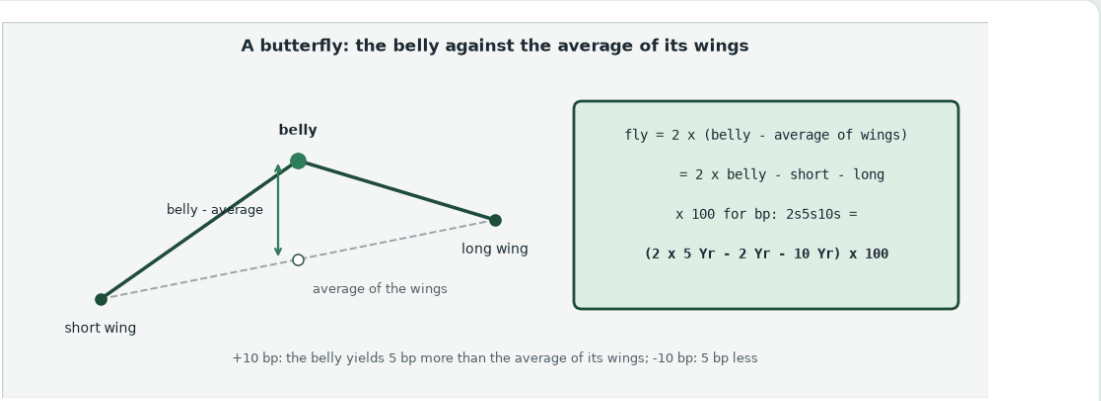


The 10 Yr sat 1.08 points under the 2 Yr. A negative slope is called inverted: a word for the shape, nothing more.

A slope is long less short, in bp: +45 bp on 2 October 2026, -108 bp at its lowest, and the sign is the direction of the line.

9.2 Butterflies: 2s5s10s and 5s10s30s

A slope sees two points; the middle of the curve needs three.



A butterfly, or fly, is twice the belly less both wings, times 100. Positive when the belly yields more than the average of its wings.

```
flies_by_hand = pd.DataFrame({
    '2s5s10s': (2 * treasury['5 Yr'] - treasury['2 Yr'] - treasury['10 Yr']) * 100,
    '5s10s30s': (2 * treasury['10 Yr'] - treasury['5 Yr'] - treasury['30 Yr']) * 100,
})
flies_by_hand.tail(3).round(2)
```

date	2s5s10s	5s10s30s
2026-09-30	1.0	-15.0
2026-10-01	-0.0	-14.0
2026-10-02	1.0	-13.0

-0.0 on 1 October is notebook 06's binary tail: a hair under zero, rounded.

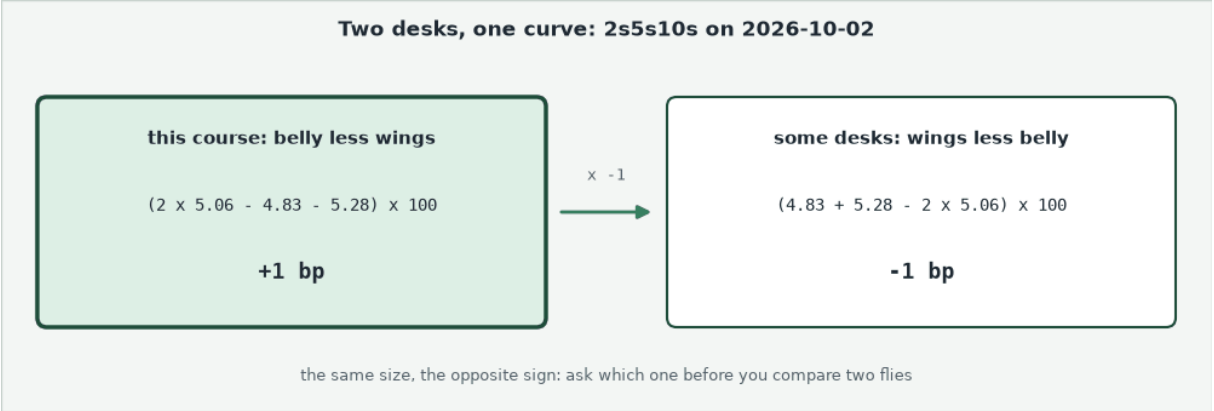
Job	Python	Excel	Result, last row
2s5s10s	`flies_by_hand['2s5s10s']`	`=(2*K2-I2-M2)*100` filled down	1
5s10s30s	`flies_by_hand['5s10s30s']`	`=(2*M2-K2-O2)*100` filled down	-13

Five numbers for 2 October 2026, each equal to Excel's on all 2,940 rows: the note's first ask.

The other sign

Some desks quote a fly the other way round: wings less belly.

```
other_sign = -flies_by_hand['2s5s10s']
print(f"2s5s10s on 2026-10-02: {flies_by_hand.loc['2026-10-02', '2s5s10s']:+.2f} bp here, "
      f"{other_sign.loc['2026-10-02']:+.2f} bp on the other sign")
2s5s10s on 2026-10-02: +1.00 bp here, -1.00 bp on the other sign
```



Ask which one before you compare. This course quotes belly less wings, everywhere.

9.3 slope_bp and butterfly_bp: the sign in the docstring

The arithmetic lives in two cells; a sign convention belongs where every reader meets it. Each docstring names the units, the convention and name order, the Excel formula, and the "ValueError"; butterfly_bp's also warns that some desks quote the other sign. "_check_index" is notebook 03's; ".rename()" gives the Series its name.

```
%%writefile -a marketdata.py
def slope_bp(curve: pd.DataFrame, short: str, long: str) -> pd.Series:
    """A curve slope in basis points: the long tenor's yield less the short tenor's.
    Units: yields in percent in, bp out. Convention: (curve[long] - curve[short]) x 100, named short then
    long: 2s10s is slope_bp(curve, "2 Yr", "10 Yr"). Positive when the longer yield is higher.
    Excel: =(M2-I2)*100 on the Treasury file opened in Excel, with the 10 Yr in M and the 2 Yr in I.
    Raises ValueError if a tenor is not a column of curve, or the index is not unique ascending dates.
    """
    _check_index(curve.index, "curve")
    for tenor in (short, long):
        if tenor not in curve.columns:
            raise ValueError(f"{tenor!r} is not a column of curve")
    return ((curve[long] - curve[short]) * 100).rename(f"{short} to {long} slope, bp")
def butterfly_bp(curve: pd.DataFrame, short: str, belly: str, long: str) -> pd.Series:
    """A butterfly in basis points: twice the belly's yield less both wings.
    Units: yields in percent in, bp out. Convention: (2 x belly - short - long) x 100, named short, belly,
    long: 2s5s10s is butterfly_bp(curve, "2 Yr", "5 Yr", "10 Yr"). Positive when the belly yields more than
    the average of the wings. Some desks quote the opposite sign.
    Excel: =(2*K2-I2-M2)*100 on the Treasury file, with the 5 Yr in K.
    Raises ValueError if a tenor is not a column of curve, or the index is not unique ascending dates.
    """
    _check_index(curve.index, "curve")
    for tenor in (short, belly, long):
        if tenor not in curve.columns:
            raise ValueError(f"{tenor!r} is not a column of curve")
    return ((2 * curve[belly] - curve[short] - curve[long]) * 100).rename(f"{short} {belly} {long} fly, bp")
```

Same numbers as by hand, and three tests

```
importlib.reload(md)
slopes = pd.DataFrame({'2s10s': md.slope_bp(treasury, '2 Yr', '10 Yr'),
                       '5s30s': md.slope_bp(treasury, '5 Yr', '30 Yr'),
                       '3m10y': md.slope_bp(treasury, '3 Mo', '10 Yr')})
flies = pd.DataFrame({'2s5s10s': md.butterfly_bp(treasury, '2 Yr', '5 Yr', '10 Yr'),
                      '5s10s30s': md.butterfly_bp(treasury, '5 Yr', '10 Yr', '30 Yr')})
print('the same as by hand:', slopes.equals(slopes_by_hand), flies.equals(flies_by_hand))
print(md.slope_bp(treasury, '2 Yr', '10 Yr').name)
the same as by hand: True True
2 Yr to 10 Yr slope, bp
```

A Tenor That Is Not There

```
md.slope_bp(treasury, '2 Yr', '10 Y')
ValueError: '10 Y' is not a column of curve
```

A positive and a negative slope, a fly worked by hand, and both functions against Excel's reference sheet: 190 rows of 2026, where the 2 Yr, 5 Yr and 10 Yr sit in B, C and D.

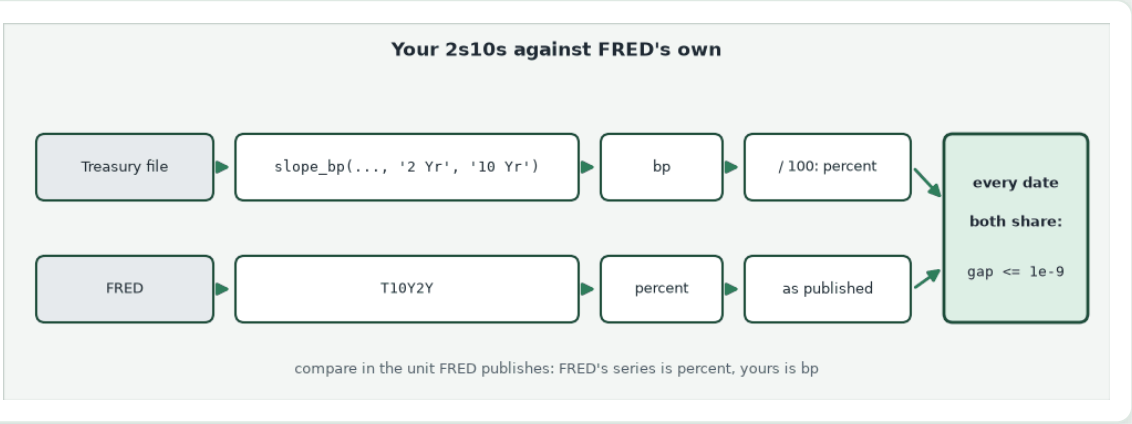
```
%%writefile -a test_marketdata.py
def test_slope_sign():
    curve = pd.DataFrame({"2 Yr": [4.88, 5.10], "10 Yr": [5.29, 5.00]}, index=pd.to_datetime(["2026-09-30", "2026-10-01"]))
    slope = md.slope_bp(curve, "2 Yr", "10 Yr")
    assert slope.iloc[0] == pytest.approx(41.0, abs=1e-9)
    assert slope.iloc[1] == pytest.approx(-10.0, abs=1e-9)
def test_butterfly_hand_value():
    curve = pd.DataFrame({"2 Yr": [4.88], "5 Yr": [5.09], "10 Yr": [5.29]}, index=pd.to_datetime(["2026-09-30"]))
    assert md.butterfly_bp(curve, "2 Yr", "5 Yr", "10 Yr").iloc[0] == pytest.approx(1.0, abs=1e-9)
def test_butterfly_matches_excel_reference():
    reference = read_reference()
    curve = read_treasury().loc[reference.index]
    fly = md.butterfly_bp(curve, "2 Yr", "5 Yr", "10 Yr")
    slope = md.slope_bp(curve, "2 Yr", "10 Yr")
    assert ((fly - reference["fly_2s5s10s_bp"]).abs() <= 1e-9).all()
    assert ((slope - reference["slope_2s10s_bp"]).abs() <= 1e-9).all()
```

```
....................................   [100%]
38 passed in 0.50s
```

The sign now lives in the docstring and the tests pin it both ways: 38 passed, equal to Excel on all 190 reference rows.

9.4 Live: FRED's T10Y2Y against your 2s10s

This product uses the FRED® API but is not endorsed or certified by the Federal Reserve Bank of St. Louis.



FRED publishes 2s10s itself as "T10Y2Y": the 10 Yr less the 2 Yr, in percent, from the same Treasury yields. An outside check on "slope_bp", in FRED's unit.

Pull and Compare

".intersection", notebook 03's, keeps the dates both series have.

```
if fred_key is None:
    print(NO_KEY_MESSAGE)
else:
    live_t10y2y = md.fred_series('T10Y2Y', fred_key, start='2015-01-01').dropna()
    print(f'T10Y2Y: {len(live_t10y2y):,} rows, the last on {live_t10y2y.index[-1]:%Y-%m-%d}')
    live_common = live_t10y2y.index.intersection(slopes.index)
    live_gap = (live_t10y2y[live_common] - slopes.loc[live_common, '2s10s'] / 100).abs().max()
    assert len(live_common) > 0 and live_gap <= 1e-9, 'T10Y2Y and your 2s10s disagree'
    print(f'{len(live_common):,} dates in both, largest gap {live_gap:.1e} percent')
Live cell skipped: no FRED key was found. Every other cell runs without one.
```

With a key, the cell prints T10Y2Y's rows since 2015 and its last date, later than the Treasury file's 2 October 2026 once FRED has published newer days, then the dates both share and the largest gap.

With a key, your 2s10s divided by 100 equals FRED's "T10Y2Y" on every date the two share; without one, nothing else in the notebook changes.

9.5 Slopes and flies over time

One number on one date; its history says where it sits.

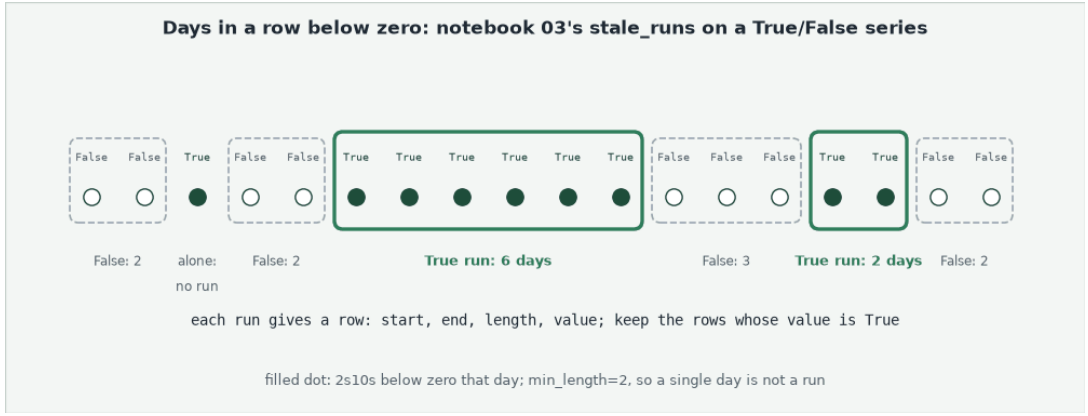
Days Below Zero

```
below_zero = slopes['2s10s'] < 0
print(f'below zero on {below_zero.sum()} of {len(slopes):,} market days; '
      f"exactly zero on {(slopes['2s10s'] == 0).sum()}")
below zero on 544 of 2,940 market days; exactly zero on 7
```

The Longest Run

Notebook 03's "stale_runs" finds runs of one repeated value. On "below_zero" a run of Trues is a stretch of days in a row below zero; it reports the runs of False too, so keep the True rows with a mask.

```
runs = md.stale_runs(below_zero, min_length=2)
negative_runs = runs[runs['value']]
longest = negative_runs.loc[negative_runs['length'].idxmax()]
print(f"{len(negative_runs)} runs below zero; the longest {longest['length']} market days, "
      f"{longest['start']:%Y-%m-%d} to {longest['end']:%Y-%m-%d}")
3 runs below zero; the longest 537 market days, 2022-07-06 to 2024-08-26
```



Excel side by side, and where 2s10s sits in its last year

Excel side by side. A sheet with dates in A, the 2 Yr in B, the 5 Yr in C, the 10 Yr in D and 2s10s in E, rows 2 to 2941; column F counts the days in a row.

Job	Python	Excel	Result
Days below zero	`below_zero.sum()`	`=COUNTIF(E2:E2941,"<0")`	544
Days exactly zero	`(slopes['2s10s'] == 0).sum()`	`=COUNTIF(E2:E2941,0)`	7
The lowest	`slopes['2s10s'].min()`	`=MIN(E2:E2941)`	-108
Its date	`lowest_date`	`=XLOOKUP(MIN(E2:E2941),E2:E2941,A2:A2941)`	45110, 2023-07-03 as a date
Days in a row below zero	`md.stale_runs(...)`	`=IF(E2<0,IF(N(E1)<0,F1+1,1),0)` in F2, filled down	a count per row
The longest run	`longest['length']`	`=MAX(F2:F2941)`	537
Its last day	`longest['end']`	`=XLOOKUP(MAX(F2:F2941),F2:F2941,A2:A2941)`	45530, 2024-08-26 as a date

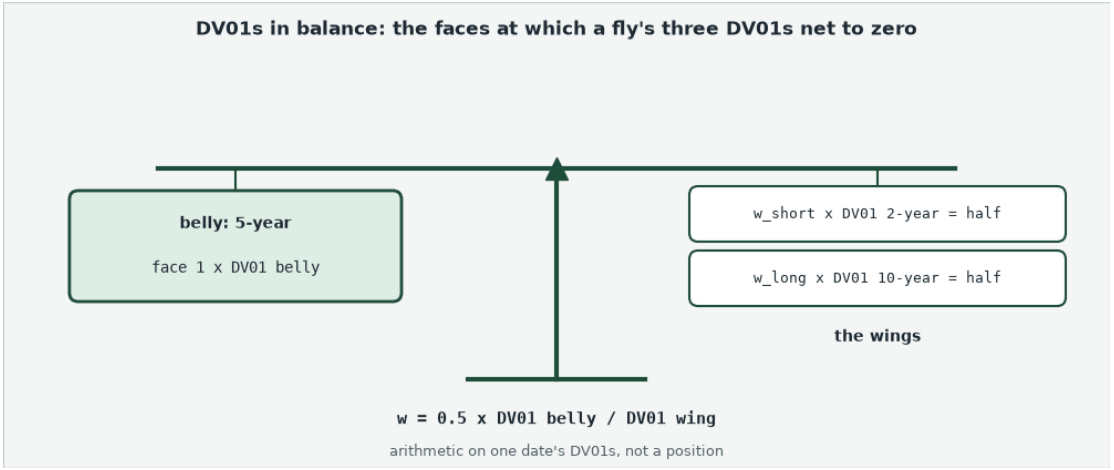
"N(E1)" turns the header's text into 0, so the formula in F2 works on the first row.

Where Each Sits in Its Last Year

```
z_2s10s = md.rolling_zscore(slopes['2s10s'])
print(f"2s10s on 2026-10-02: {slopes['2s10s'].iloc[-1]:.2f} bp, z-score {z_2s10s.iloc[-1]:.2f}")
2s10s on 2026-10-02: 45.00 bp, z-score -0.51
```

9.6 DV01-neutral fly weights, from your bondmath

A fly in bp compares yields. Three bonds move by different amounts for the same 1 bp: Course 2's DV01.



The belly's face is 1. Each wing's face is set so it carries half the belly's DV01: w = 0.5 x DV01 of the belly / DV01 of the wing.

These are the faces at which three DV01s net to zero on one date: arithmetic, not a position.

Three Par Bonds

A par bond at each tenor: the coupon equal to that tenor's yield on 2 October 2026, settling that day, maturing 2, 5 or 10 years on. On a coupon date accrued interest is 0, so each prices at 100.

```
from datetime import date
import bondmath
settlement = date(2026, 10, 2)
```

```
dv01_by_tenor = {}
for tenor, years in [('2 Yr', 2), ('5 Yr', 5), ('10 Yr', 10)]:
    yield_pct = treasury.loc['2026-10-02', tenor]
    maturity = bondmath.add_months(settlement, 12 * years)
    dv01_by_tenor[tenor] = bondmath.dv01(settlement, maturity, yield_pct, yield_pct)
    clean = bondmath.price(settlement, maturity, yield_pct, yield_pct)
    print(f'{tenor}: price {clean:.6f}, DV01 {dv01_by_tenor[tenor]:.6f} per 100 of face per bp')
2 Yr: price 100.000000, DV01 0.018848 per 100 of face per bp
5 Yr: price 100.000000, DV01 0.043693 per 100 of face per bp
10 Yr: price 100.000000, DV01 0.076925 per 100 of face per bp
```

Excel side by side, then fly_weights

Excel side by side. One column per bond, B the 2-year, C the 5-year, D the 10-year: "PRICE" in row 1, the accrued interest in row 2, "MDURATION" in row 3, the DV01 in row 4.

Job	Python	Excel	Result
5-year price	`bondmath.price(...)`	`=PRICE(DATE(2026,10,2),DATE(2031,10,2),5.06/100,5.06/100,100,2,0)`	100.000000
5-year accrued	`0 on a coupon date`	`=100*5.06/100/2*COUPDAYBS(...)/COUPDAYS(...)`	0
5-year modified duration	`bondmath.modified_duration(...)`	`=MDURATION(DATE(2026,10,2),DATE(2031,10,2),5.06/100,5.06/100,2,0)`	4.369256
5-year DV01	`dv01_by_tenor['5 Yr']`	`=C3*(C1+C2)/10000 in C4`	0.043693
2-year and 10-year DV01	`dv01_by_tenor['2 Yr'], ['10 Yr']`	`the same in B4 and D4`	0.018848, 0.076925

The docstring names the units, the convention, the order of the three faces it returns, and the "ValueError" for a DV01 that is not positive.

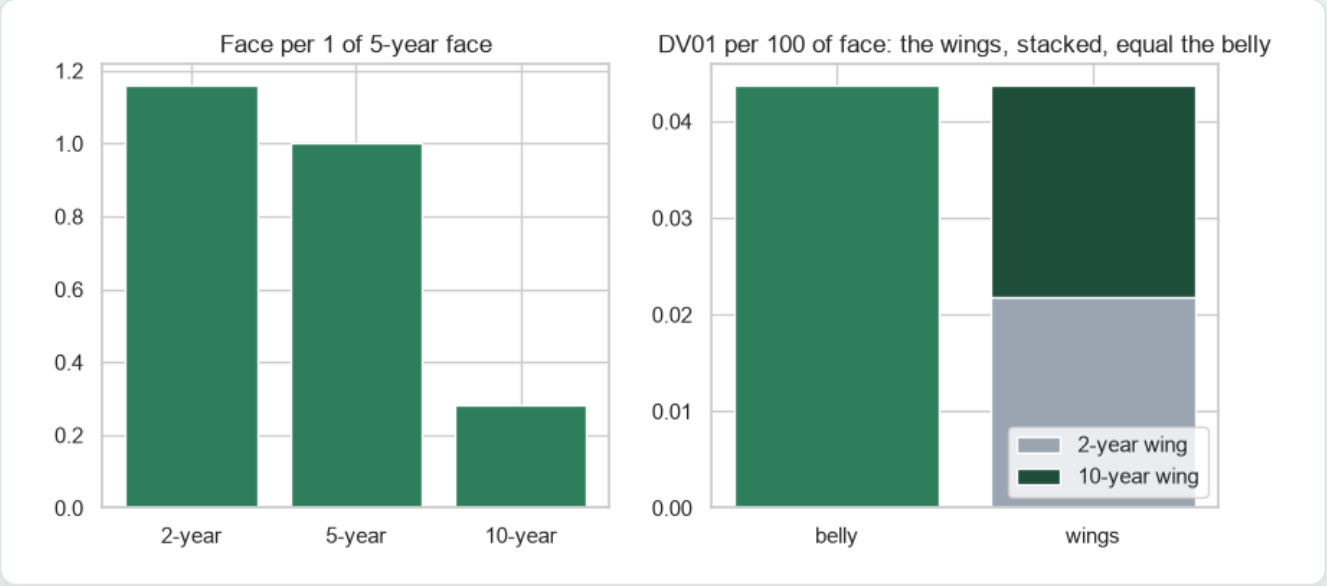
```
%%writefile -a marketdata.py
def fly_weights(dv01_short: float, dv01_belly: float, dv01_long: float) -> tuple[float, float, float]:
    """Face amounts for a DV01-neutral butterfly, per 1 of belly face.
    Units: DV01s in any one unit (price points per 100 of face per bp from bondmath.dv01); weights are face
    per 1 of belly face. Convention: the belly's face is 1; each wing's face is set so its DV01 is half the
    belly's: w_short = 0.5 x dv01_belly / dv01_short and w_long = 0.5 x dv01_belly / dv01_long. With the
    belly on one side and both wings on the other, the DV01s net to zero. Arithmetic, not a position.
    Returns (w_short, 1.0, w_long). Raises ValueError if a DV01 is not positive.
    """
    for label, dv01 in (("dv01_short", dv01_short), ("dv01_belly", dv01_belly), ("dv01_long", dv01_long)):
        if dv01 <= 0:
            raise ValueError(f"{label} must be positive, not {dv01}")
    w_short = 0.5 * dv01_belly / dv01_short
    w_long = 0.5 * dv01_belly / dv01_long
    return w_short, 1.0, w_long
```

The faces, and the net

```
importlib.reload(md)
w_short, w_belly, w_long = md.fly_weights(dv01_by_tenor['2 Yr'], dv01_by_tenor['5 Yr'],
                                          dv01_by_tenor['10 Yr'])
print(f'faces per 1 of 5-year face: 2-year {w_short:.6f}, 10-year {w_long:.6f}')
short_dv01, long_dv01 = w_short * dv01_by_tenor['2 Yr'], w_long * dv01_by_tenor['10 Yr']
net = w_belly * dv01_by_tenor['5 Yr'] - short_dv01 - long_dv01
print(f"belly {dv01_by_tenor['5 Yr']:.6f} = {short_dv01:.6f} + {long_dv01:.6f}; net {net:.1e}")
faces per 1 of 5-year face: 2-year 1.159049, 10-year 0.283995
belly 0.043693 = 0.021846 + 0.021846; net 0.0e+00
```

Excel side by side. The weights in B6 and C6, from the DV01s in row 4.

Job	Python	Excel	Result
2-year face	`w_short`	`=0.5*C4/B4`	1.159049
10-year face	`w_long`	`=0.5*C4/D4`	0.283995
The net	`net`	`=C4-B6*B4-C6*D4`	0



Two tests

The test's DV01s are rounded values near today's, so it pins the arithmetic, not a date.

```
%%writefile -a test_marketdata.py
def test_fly_weights_neutral():
    dv01_short, dv01_belly, dv01_long = 0.0186, 0.0436, 0.0778
    w_short, w_belly, w_long = md.fly_weights(dv01_short, dv01_belly, dv01_long)
    assert w_belly == 1.0
    assert w_belly * dv01_belly - w_short * dv01_short - w_long * dv01_long == pytest.approx(0.0, abs=1e-15)
    assert w_short * dv01_short == pytest.approx(w_long * dv01_long, abs=1e-15)
def test_fly_weights_reject_nonpositive():
    with pytest.raises(ValueError):
        md.fly_weights(0.0186, 0.0, 0.0778)
    with pytest.raises(ValueError):
        md.fly_weights(-0.0186, 0.0436, 0.0778)
```

```
........................................   [100%]
40 passed in 0.50s
```

On 2 October 2026, 1.159049 of 2-year and 0.283995 of 10-year face against 1 of 5-year net the three DV01s to zero, as Excel's "MDURATION" and "PRICE" give: arithmetic, not a position.

9.7 2026's slopes as a data file under Git

The note's last ask: a rerun that moves a number should show up. Git shows it, once the numbers sit in a file it tracks.

```
!git -C "{work}" status --short
!git -C "{work}" diff --stat
 M marketdata.py
 M test_marketdata.py
 marketdata.py   | 52 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 test_marketdata.py | 41 +++++++++++++++++++++++++++++++++++++++++
 2 files changed, 93 insertions(+)
```

"float_format='%.2f'" writes every number with two decimals; "lineterminator='\n'" is notebook 05's.

```
slopes.loc['2026'].to_csv('slopes_2026.csv', float_format='%.2f', lineterminator='\n')
with open('slopes_2026.csv') as file:
    lines = file.readlines()
print(len(lines) - 1, 'rows under the header')
print(''.join(lines[:3]))
190 rows under the header
date,2s10s,5s30s,3m10y
2026-01-02,72.00,112.00,54.00
2026-01-05,71.00,114.00,53.00
```

```
subject = 'Notebook 09: slope_bp, butterfly_bp, fly_weights, and slopes_2026.csv'
!git -C "{work}" add marketdata.py test_marketdata.py slopes_2026.csv
!git -C "{work}" commit -q -m "{subject}"
!git -C "{work}" log --oneline -2
19940f9 Notebook 09: slope_bp, butterfly_bp, fly_weights, and slopes_2026.csv
de7069f Notebook 08: rolling_zscore and rolling_percentile_rank
```

Q. A rerun flips one sign by mistake. Does Git show it?

```
edited = pd.read_csv('slopes_2026.csv', index_col='date')
edited.loc['2026-09-30', '2s10s'] = -41.0
edited.to_csv('slopes_2026.csv', float_format='%.2f', lineterminator='\n')
!git -C "{work}" diff slopes_2026.csv
diff --git a/slopes_2026.csv b/slopes_2026.csv
index 38b2fe8..08d6126 100644
--- a/slopes_2026.csv
+++ b/slopes_2026.csv
@@ -186,6 +186,6 @@ date,2s10s,5s30s,3m10y
 2026-09-25,36.00,51.00,93.00
 2026-09-28,32.00,50.00,96.00
 2026-09-29,37.00,53.00,101.00
-2026-09-30,41.00,55.00,109.00
+2026-09-30,-41.00,55.00,109.00
 2026-10-01,46.00,60.00,107.00
 2026-10-02,45.00,57.00,109.00
```

One "-" line, the committed 41.00, and one "+" line, the slip: a hunk read as in Course 2's Cash Flows and Price From Yield.

```
!git -C "{work}" restore slopes_2026.csv
!git -C "{work}" status --short
print(pd.read_csv('slopes_2026.csv', index_col='date').loc['2026-09-30', '2s10s'])
41.0
```

Bring it home, and what you can now do

Copy the work folder's two files and "slopes_2026.csv" into your own "my_bondmath" folder, test them there, and commit. The copy replaces your files of those names, so anything of your own must be in the work folder first. The tests read the Treasury file and Excel's reference file, in your folder since notebooks 04 and 06.

```
$work = "$env:TEMP\pfi_course3_09_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py", "$work\slopes_2026.csv" .
python -m pytest -q test_marketdata.py
git status --short
git add marketdata.py test_marketdata.py slopes_2026.csv
git commit -m "Notebook 09: slope_bp, butterfly_bp, fly_weights, and slopes_2026.csv"
```

pytest shows 40 passed; "git status --short" shows the two files changed and "slopes_2026.csv" new. In Colab, download the three files from the work folder under "/tmp" instead. The work folder stays in your temp folder; the system clears it in time.

You Can Now

Compute a slope as long less short in bp, and read its sign as the direction of the curve.

Compute a butterfly as twice the belly less both wings, and convert from the other desk's sign.

Write a sign convention into a docstring and pin it with tests tied to Excel.

Check your 2s10s against FRED's "T10Y2Y" in the unit FRED publishes.

Count days and runs below zero, and say where a slope or fly sits in its own last year.

Compute DV01-neutral fly weights from "bondmath", as arithmetic, not a position.

Keep a data file under Git, read a changed number in the diff, and restore it.

Predict first: the fly on 3 July 2023, and days below zero

Predict 1

On 3 July 2023, the lowest 2s10s, the 2 Yr was 4.94 and the 10 Yr 3.86 percent. What was 2s5s10s?

A) strongly positive

B) close to zero

C) strongly negative

```
print(f"5 Yr {treasury.loc[lowest_date, '5 Yr']:.2f} percent; "
      f"2s5s10s {flies_by_hand.loc[lowest_date, '2s5s10s']:.2f} bp")
```

Predict 2

2s10s was below zero on 544 of 2,940 market days. Which other slope spent more days below zero?

A) 5s30s

B) 3m10y

C) neither

```
(slopes < 0).sum()
```

Predict first: the fly's z-score, a wing, a format

Predict 3

The 2s5s10s fly is +1 bp on 2 October 2026, close to zero. How many standard deviations from the mean of its last 252 market days does it sit? Write down a number.

```
z_fly = md.rolling_zscore(flies['2s5s10s'])
fly_window = flies['2s5s10s'].iloc[-252:]
print(f"2s5s10s: {flies['2s5s10s'].iloc[-1]:.2f} bp, z-score {z_fly.iloc[-1]:.2f}")
print(f'its last 252 days: mean {fly_window.mean():.2f} bp, high {fly_window.max():.2f} bp')
```

Predict 4

Move all three yields up 1 point, still par bonds. The 2-year wing's face of 1.1590:

A) falls

B) rises

C) stays the same

```
shifted = {}
for tenor, years in [('2 Yr', 2), ('5 Yr', 5), ('10 Yr', 10)]:
    yield_pct = treasury.loc['2026-10-02', tenor] + 1
    maturity = bondmath.add_months(settlement, 12 * years)
    shifted[tenor] = bondmath.dv01(settlement, maturity, yield_pct, yield_pct)
shifted_weights = md.fly_weights(shifted['2 Yr'], shifted['5 Yr'], shifted['10 Yr'])
print(f'2-year wing {w_short:.4f} to {shifted_weights[0]:.4f}; '
      f'10-year wing {w_long:.4f} to {shifted_weights[2]:.4f}')
```

Predict 5

Write the same slopes with '%.1f' in place of '%.2f': no value changes. How many lines does Git count as changed?

A) none

B) a few

C) every data row

```
slopes.loc['2026'].to_csv('slopes_2026.csv', float_format='%.1f', lineterminator='\n')
!git -C "{work}" diff --stat
!git -C "{work}" restore slopes_2026.csv
```

The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

Docstring

Units, conventions, and what it must never do: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: "return (2 * curve[belly] - (curve[short] - curve[long])) * 100" adds the long wing back: not the hand value.

Code that runs is not the same as code that is right.

The note, answered

The note asked	Answer
Five shapes, signs written down	2s10s +45, 5s30s +57, 3m10y +109, 2s5s10s +1, 5s10s30s -13 bp; flies belly less wings
In marketdata, tied to Excel	slope_bp and butterfly_bp, equal on 2,940 rows; 38 then 40 passed
FRED's own 2s10s	T10Y2Y equal on every common date, with your key
The fly's faces	1.159049 of 2-year and 0.283995 of 10-year per 1 of 5-year
2026 under Git	slopes_2026.csv committed with the module, a slip read in the diff and restored

Be proud: Course 2's bond math and this course's market series worked in one notebook, and every sign was written down before it was used.

Next is notebook 10, Case Study: Three Years of Curve and Spreads, where the week's measures go into one table.